



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

LABORATORIO DE COMPUTACIÓN GRÁFICA E
INTERACCIÓN HUMANO COMPUTADORA

PRÁCTICA 01

Grupo de laboratorio: 06

Profesor: Dr. Sergio Teodoro Vite

Integrantes:

- Basilio Illescas Andrés
- Cruz Macedo Samuel Santiago
- Medel Piña Ángel de Jesús
- Pérez Olvera Alexis Abraham
- Segura Cedeño Luisa María



Fecha de entrega: Martes 01 de septiembre del 2026

Semestre: 2027-1

INTRODUCCIÓN A OPENGL

- 321137135 andres.basilio@ingenieria.unam.edu
- 424065465 santi.sam120@gmail.com
- 321258173 angelmedel130105@gmail.com
- 321150503 ralexisabrahamr@gmail.com
- 320224159 segura.cedeno.luisa@gmail.com

1. Resumen

Resumen

En este reporte se presentan los resultados de la primera práctica de laboratorio centrada en la introducción a la API gráfica OpenGL. El objetivo principal fue comprender el flujo de trabajo básico para el renderizado en 2D, abarcando desde la configuración inicial de la ventana y el contexto de dibujo, hasta la interacción con el hardware mediante interrupciones de teclado. A nivel de implementación, se trabajó con la definición de geometrías mediante arreglos y búferes de vértices (VAO y VBO), así como la asignación dinámica de colores normalizados en formato de punto flotante. Como resultado, se desarrollaron rutinas interactivas capaces de cambiar el color de fondo en tiempo real y se logró el despliegue en pantalla de primitivas gráficas y funciones matemáticas, incluyendo un rectángulo multicolor, un medio círculo, un pentágono y la función tangente. Estas actividades permitieron afianzar las bases de la comunicación entre el software y la tarjeta gráfica para futuros proyectos de la asignatura.

Abstract

This report presents the results of the first laboratory practice focused on the introduction to the OpenGL graphics API. The main objective was to understand the basic workflow for 2D rendering, ranging from the initial configuration of the window and drawing context to hardware interaction through keyboard callbacks. At the implementation level, we worked with the definition of geometries using vertex arrays and buffers (VAO and VBO), as well as the dynamic assignment of normalized floating-point colors. As a result, interactive routines were developed capable of changing the background color in real-time, successfully displaying graphic primitives and mathematical functions on screen, including a multicolored rectangle, a half-circle, a pentagon, and the tangent function. These activities solidified the foundations of software-to-graphics-card communication for future projects in this course.

2. Introducción

OpenGL actúa como un puente entre el software y la tarjeta gráfica que es el hardware. Su objetivo principal es tomar información como puntos (vértices) o imágenes (píxeles) y convertirla en la figura final que se observa en pantalla. Para entender su funcionamiento, hay que saber que se apoya en conceptos básicos como el uso de figuras (puntos, líneas y polígonos), reglas sobre qué tareas gráficas puede y no puede controlar, un modelo de comunicación cliente-servidor, un proceso paso a paso para generar la imagen y una forma ordenada y estricta de nombrar sus funciones.

En lugar de crear modelos 3D elaborados, OpenGL trabaja de la siguiente forma: se le indica exactamente cada paso para manipular la posición, el color y la iluminación de los objetos. Funciona con un modelo de cliente y servidor, donde el código del usuario envía órdenes y OpenGL las ejecuta. Las tareas externas, como abrir la ventana del programa o capturar las teclas y clics del ratón, quedan a cargo del sistema operativo.

El proceso para generar una imagen en pantalla sigue tres etapas clave:

- **Vértices:** Procesa y ubica los puntos de las figuras geométricas básicas.
- **Rasterización:** Transforma esas figuras o imágenes en fragmentos de pantalla.
- **Fragmentos:** Ajusta qué elementos van al frente o atrás y define el color final de cada píxel.

Finalmente, los comandos de OpenGL comparten una estructura estándar y cambian su terminación según el tipo y número de datos que reciben (por ejemplo, `glVertex2f`). Además, muchas de sus funciones visuales se activan o desactivan de manera manual utilizando instrucciones como `glEnable` y `glDisable`.

3. Metodología de la practica

3.1. Actividad 1: Cambio de color de fondo

Para resolver el primer ejercicio, se requirio la conversión del formato de color. A diferencia de las herramientas de diseño convencionales, OpenGL no interpreta los colores en hexadecimal ni en la escala RGB de 0 a 255, sino que requiere valores de punto flotante normalizados en un rango de 0.0f a 1.0f.

El algoritmo de conversión matemática que aplicamos para la práctica consistió en dos pasos elementales:

- Separar el código hexadecimal de la paleta en sus componentes independientes (rojo, verde y azul) y transformarlos a base decimal.
- Dividir cada uno de estos valores decimales entre 255.0 para obtener su equivalente normalizado como un número flotante.

Tomando como ejemplo el primer color solicitado, **#00C2CB**, el canal rojo (00) se mantiene en 0.0f. El canal verde (C2) equivale a 194 en decimal, que dividido entre 255 nos da 0.7607f. Finalmente, el canal azul (CB) es 203, arrojando un valor de 0.7960f. Iteramos esta misma rutina matemática para extraer los valores de los cinco tonos de la paleta exigida.

A nivel de arquitectura de software, trabajamos sobre el archivo principal `01_IntroOpenGL.cpp`. Para que el color reaccionara en tiempo de ejecución, nos enganchamos a la función `keyboardCallback` del código base, la cual gestiona las interrupciones de hardware.

Tal como lo indica la rúbrica de evaluación, a continuación se presenta el segmento de código relevante con la implementación. Mapeamos las teclas R, G, B, Y, y M para sobrescribir dinámicamente las variables globales `bgR`, `bgG`, `bgB` y `bgA`. En cada ciclo de fotograma, la función `Update()` lee estas variables y se las pasa al comando nativo `glClearColor`, logrando así el barrido y pintado de la pantalla con el color seleccionado:

Código de los cambios realizados para el cambio de color de fondo:

```
1      // Actividad 1.1
2      if (glfwGetKey(window, GLFW_KEY_R) == GLFW_PRESS) {
3          bgR = 0.0f;
4          bgG = 0.7607f;
5          bgB = 0.7960f;
6          bgA = 1.0;
7      }
8      if (glfwGetKey(window, GLFW_KEY_G) == GLFW_PRESS) {
9          bgR = 0.4980f;
10         bgG = 0.8784f;
11         bgB = 0.8313f;
12         bgA = 1.0;
13     }
14     if (glfwGetKey(window, GLFW_KEY_B) == GLFW_PRESS) {
15         bgR = 1.0f;
16         bgG = 0.8901f;
17         bgB = 0.5411f;
18         bgA = 1.0;
19     }
20     if (glfwGetKey(window, GLFW_KEY_N) == GLFW_PRESS) {
21         bgR = 1.0f;
22         bgG = 0.6039f;
23         bgB = 0.4627f;
24         bgA = 1.0;
25     }
26     if (glfwGetKey(window, GLFW_KEY_O) == GLFW_PRESS) {
```

```

27         bgR = 1.0f;
28         bgG = 0.3647f;
29         bgB = 0.5607f;
30         bgA = 1.0;
31     }

```

Listing 1: Código del ejercicio 1

3.2. Actividad 2: Rectángulo unido por dos triángulos

Para realizar esta práctica, el trabajo se dividió en cinco actividades; primero, se preparó el entorno de trabajo. Se abrió la ventana donde se iba a mostrar todo usando herramientas como GLFW y GLAD, y se ajustó el tamaño del área de dibujo con `glViewport`. También se dejaron listas las funciones para que el programa pudiera responder cuando el usuario presionara una tecla o moviera el ratón.

En el segundo paso, se trabajó con el color de fondo de la ventana. Como los colores venían en código hexadecimal, primero se convirtieron a números decimales del 0 al 1. Con estos valores listos, se usó la función `glClearColor` para probar y cambiar el fondo con cada uno de los colores solicitados en la paleta.

Después, se configuró la parte interna de los gráficos creando los *shaders* (el de vértices y el de fragmentos), que son pequeños códigos que se encargan de la posición y el color de las figuras. Se compilaron y se prepararon las estructuras de memoria (los VAO y VBO) para enviar las coordenadas y los colores desde la computadora hacia la tarjeta gráfica.

Por último, se hicieron pruebas y una pequeña variación a la figura y ejercicios anteriores, cambiando los colores y su rotación, para comprobar cómo respondía el programa y registrar todos los resultados obtenidos.

Código de los cambios realizados para realizar el rectángulo:

```

1  #ifndef TRIANGLE_H
2  #define TRIANGLE_H
3
4  #include <glad/glad.h>
5  #include <glm/glm.hpp>
6  #include <glm/gtc/matrix_transform.hpp>
7  #include "Vertex.h"
8  #include <string>
9  #include <fstream>
10 #include <sstream>
11 #include <iostream>
12 #include <vector>
13
14 using namespace std;
15 using namespace glm;
16
17 class Triangle {
18 public:
19     Triangle(float size) {
20         buildVertices(size);
21         buildIndices();
22     }
23
24     ~Triangle() {
25         vertices.clear();
26         indices.clear();
27     }
28
29     vector<Vertex> vertices;
30     vector<unsigned int> indices;
31

```

```

32 private:
33
34 void buildVertices(float size) {
35
36     Vertex v1, v2, v3, v4;
37
38     // Vertice 1: #FF5A5F (Coral)
39     v1.Position.x = -0.6f; v1.Position.y = 0.3f; v1.Position.z = 0.0f;
40     v1.Color.x = 1.0f;      v1.Color.y = 0.353f;    v1.Color.z = 0.373f;
41
42     // Vertice 2: #FFB400 (Amarillo)
43     v2.Position.x = -0.6f; v2.Position.y = -0.3f; v2.Position.z = 0.0f;
44     v2.Color.x = 1.0f;      v2.Color.y = 0.706f;    v2.Color.z = 0.0f;
45
46     // Vertice 3: #3DDC84 (Verde)
47     v3.Position.x = 0.6f; v3.Position.y = -0.3f; v3.Position.z = 0.0f;
48     v3.Color.x = 0.239f;   v3.Color.y = 0.863f;    v3.Color.z = 0.518f;
49
50     // Vertice 4: #00A6ED (Azul)
51     v4.Position.x = 0.6f; v4.Position.y = 0.3f; v4.Position.z = 0.0f;
52     v4.Color.x = 0.0f;      v4.Color.y = 0.651f;    v4.Color.z = 0.929f;
53
54     vertices.push_back(v1);
55     vertices.push_back(v2);
56     vertices.push_back(v3);
57     vertices.push_back(v4);
58 }
59
60 void buildIndices() {
61
62     // Triangulo 1
63     indices.push_back(0);
64     indices.push_back(1);
65     indices.push_back(2);
66
67     // Triangulo 2
68     indices.push_back(0);
69     indices.push_back(2);
70     indices.push_back(3);
71 }
72 };
73
74 #endif

```

Listing 2: Implementación de la clase Triangle para la Actividad 2 (Triangle.h)

3.3. Actividad 3: Medio círculo relleno

Para esta actividad se comentó la construcción del triángulo dentro de la función **Start()** y se habilitó en su lugar la instancia de la clase **Circle**, ajustando en consecuencia los argumentos con los que se construye el objeto **Mesh**.

```

1 // Init geometries
2 // Activity 1.2
3 // Triangle tri(0.5f);           // <-- se comento
4
5 // Activity 1.3-1.5
6 Circle circle(0.5f);           // <-- se descomento
7
8 mesh = new Mesh(circle.vertices, circle.indices); // <-- MODIFICADO
9 if (mesh == nullptr) {
10     cout << "Error creating mesh object." << endl;

```

```

11     return false;
12 }

```

Listing 3: Inicialización de la geometría en la función `Start()`.

La clase `Circle` construye su geometría tomando como primer vértice el centro de la figura y recorriendo después el ángulo `theta` en incrementos fijos, de modo que cada punto del contorno se obtiene evaluando $x = size \cdot \cos \theta$ y $y = size \cdot \sin \theta$. Los vértices quedan así ordenados de forma consecutiva alrededor del centro, lo cual determina la primitiva de dibujo que debe emplearse.

Al ejecutar el programa con la configuración original, la figura aparecía fragmentada. Esto se debe a que `GL_TRIANGLES` agrupa los vértices de tres en tres de manera independiente, criterio que no corresponde con el orden en que la clase los genera. Por ello, en el método `Draw()` de la clase `Mesh` se sustituyó dicha primitiva por `GL_TRIANGLE_FAN`, que interpreta el primer vértice como pivote y forma un triángulo entre él y cada par de vértices contiguos del contorno, generando un abanico que rellena la superficie completa.

```

1 if (indices.size() > 0)
2     glDrawElements(GL_TRIANGLE_FAN, (GLsizei)indices.size(),
3                   GL_UNSIGNED_INT, 0);           // <-- MODIFICADO
4 else if (vertices.size() > 0)
5     glDrawArrays(GL_LINE_STRIP, 0, (GLsizei)vertices.size());

```

Listing 4: Cambio de primitiva de dibujo en el método `Draw()` de la clase `Mesh`.

Finalmente, dado que la actividad solicita únicamente medio círculo, en el archivo `Circle.h` se modificó el límite del recorrido angular de 360° a 180° , con lo que el trazo se detiene a la mitad de la circunferencia.

```

1 for (int theta = 0; theta <= 180; theta += 15) { // <-- MODIFICADO

```

Listing 5: Modificación del recorrido angular en `Circle.h`.

Al conservar el paso de 15° , el semicírculo queda conformado por doce triángulos a partir de trece vértices de contorno más el vértice central. El resultado se muestra en la Figura ??.

=====

4. Experimentos

4.1. Actividad 1: Cambio de color de fondo (`glClearColor`)

Para resolver este primer ejercicio, el objetivo fue aplicar una paleta de cinco colores específicos al fondo de nuestra ventana de renderizado. Dado que OpenGL trabaja con valores flotantes normalizados entre 0.0 y 1.0 en lugar del formato hexadecimal tradicional, el primer paso de la implementación fue realizar la conversión matemática de los colores solicitados dividiendo sus valores RGB entre 255.

Una vez obtenidos los porcentajes de intensidad para cada canal (rojo, verde y azul), trabajamos directamente sobre el código fuente en el archivo `01_IntroOpenGL.cpp`. Nos enfocamos en la función `keyboardCallback`, que es la encargada de manejar las interrupciones generadas por el hardware de entrada. Dentro de esta función, aprovechamos las sentencias de control asociadas a las teclas R, G, B, N, y O. En cada bloque condicional, reasignamos los valores calculados a las variables globales `bgR`, `bgG`, `bgB` y `bgA`.

La lógica detrás de esta modificación es que, al presionar una tecla, el callback actualiza instantáneamente las variables de color globales. Posteriormente, durante el ciclo continuo de la función `Update()`, la API invoca a `glClearColor` utilizando estos nuevos valores, lo que limpia el búfer de la

pantalla y dibuja el nuevo tono de fondo en el siguiente fotograma.

A continuación, se documentan las pruebas de ejecución para cada uno de los colores configurados:

En la siguiente figura se muestra el resultado al presionar la tecla 'R', donde el fondo adopta el color hexadecimal #00C2CB.

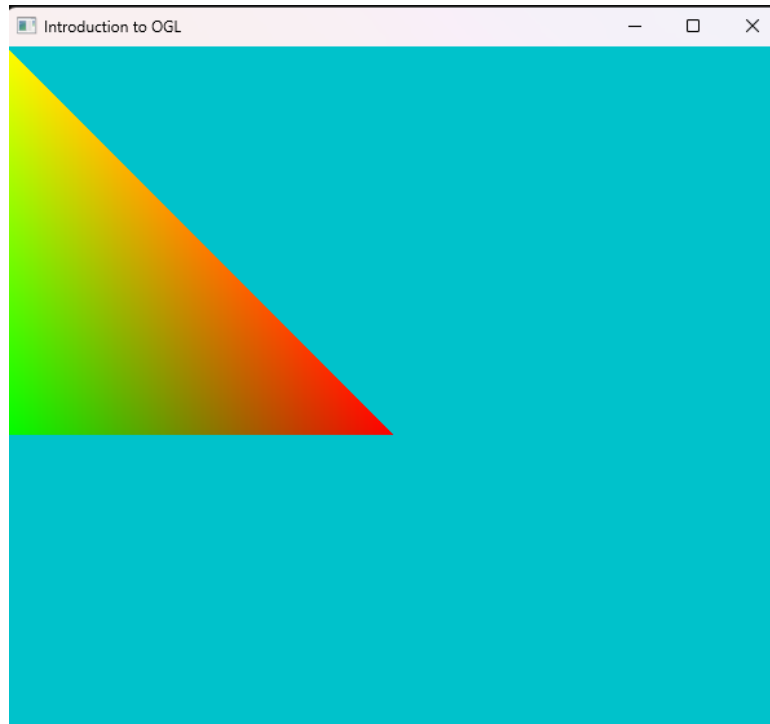


Figura 1: Ventana OpenGL con color de fondo #00C2CB.

En esta figura se observa el cambio al color #7FE0D4 (tecla 'G').



Figura 2: Ventana OpenGL con color de fondo #7FE0D4.

En esta figura se observa el cambio al color #FFE38A (tecla 'B').

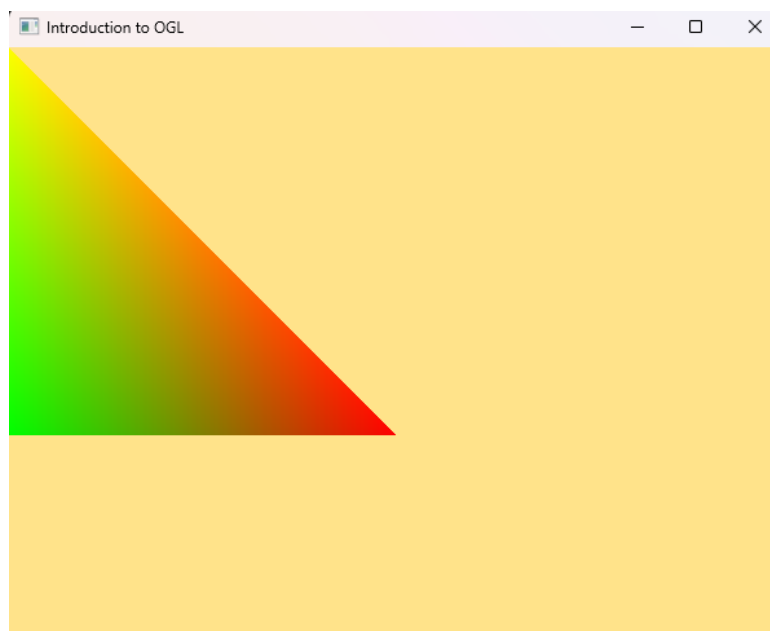


Figura 3: Ventana OpenGL con color de fondo #FFE38A.

En esta figura se observa el cambio al color #FF9A76 (tecla 'N').

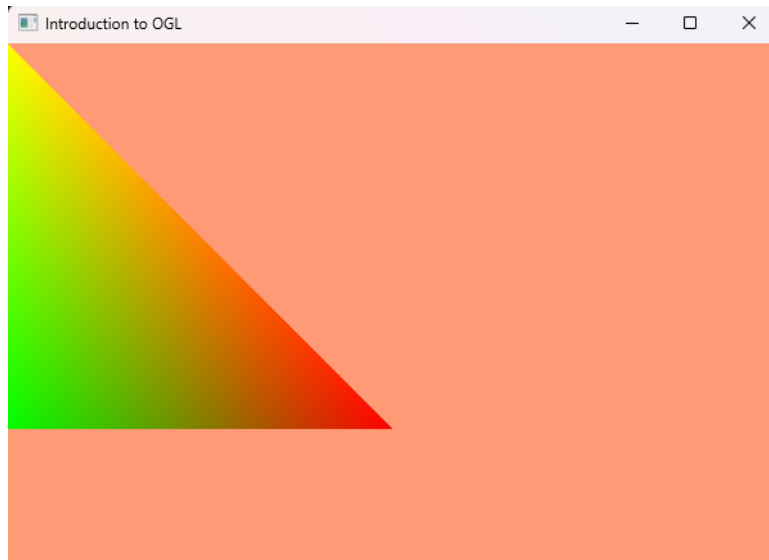


Figura 4: Ventana OpenGL con color de fondo #FF9A76.

Y por último, en la siguiente figura se observa el cambio al color #FF5D8F (tecla 'O').



Figura 5: Ventana OpenGL con color de fondo #FF5D8F.

4.2. Actividad 2: Rectángulo unido por dos triángulos

Para esta actividad, se modificó la clase encargada de construir la geometría para generar un rectángulo centrado en la ventana. Se definieron cuatro vértices cuyas coordenadas en X y en Y se calcularon de forma simétrica a partir del origen $(0,0,0,0)$, asegurando que la figura mantuviera una posición uniforme en el espacio normalizado. Para completar la superficie rectangular, se definieron dos triángulos en el arreglo de índices (**EBO**): el primero uniendo los vértices $0 - 1 - 2$ y el segundo conectando los vértices $0 - 2 - 3$.

A cada uno de los cuatro vértices se le asignó un color diferente de la paleta solicitada, realizando la conversión de sus valores hexadecimales al formato de punto flotante normalizado en el rango $[0,0,1,0]$:

- Vértice 0 (Superior izquierdo): $\text{\#FF5A5F} \rightarrow (1,000, 0,353, 0,373)$
- Vértice 1 (Inferior izquierdo): $\text{\#FFB400} \rightarrow (1,000, 0,706, 0,000)$
- Vértice 2 (Inferior derecho): $\text{\#3DDC84} \rightarrow (0,239, 0,863, 0,518)$
- Vértice 3 (Superior derecho): $\text{\#00A6ED} \rightarrow (0,000, 0,651, 0,929)$

Al momento de ejecutar el programa, el *Fragment Shader* generó una interpolación bilineal por toda la superficie del rectángulo, mostrando la transición entre los cuatro colores. A continuación, se muestra el resultado.

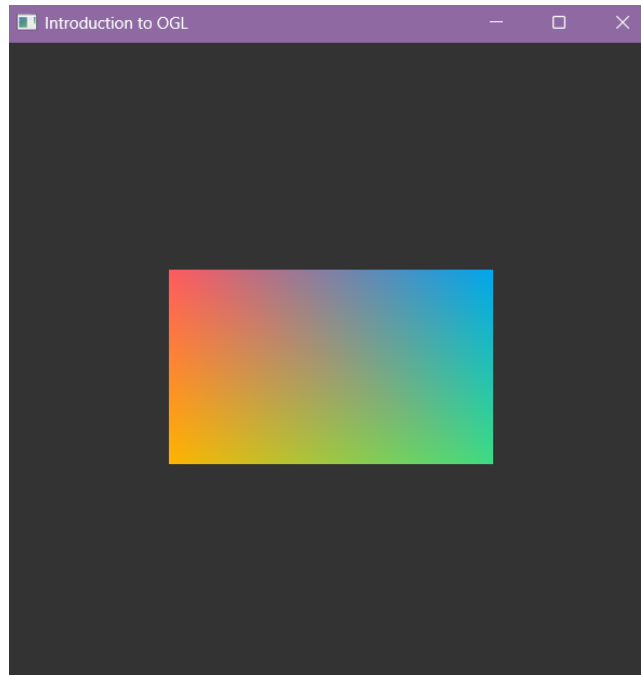


Figura 6: Ventana OpenGL con el rectángulo.

Ahora se muestra el mismo rectángulo pero con el fondo de color azul:

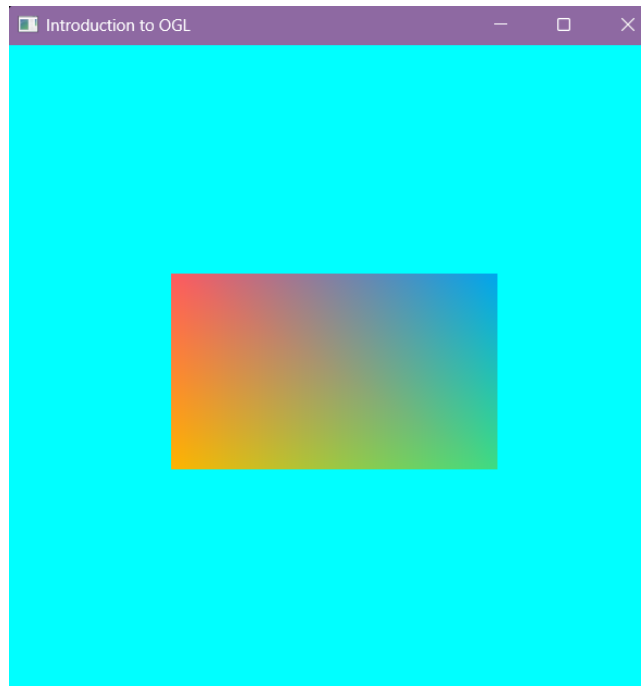


Figura 7: Ventana OpenGL con el rectángulo y fondo azul.

4.3. Actividad 2.1: Rectángulo unido por dos triángulos (Variación)

Como variación, se adaptaron las dimensiones del rectángulo para que se proyectara con una orientación vertical en lugar de horizontal, conservando su posición centrada en $(0,0,0,0)$. Asimismo, se cambió el color del vértice superior derecho por el tono morado ($\#8B5CF6 \rightarrow (0,545,0,361,0,965)$), integrando así el quinto color de la paleta. Adicionalmente, se modificó el orden de conexión de los índices a $0-1-3$ y $1-2-3$ para cambiar la dirección de la diagonal compartida por ambos triángulos.

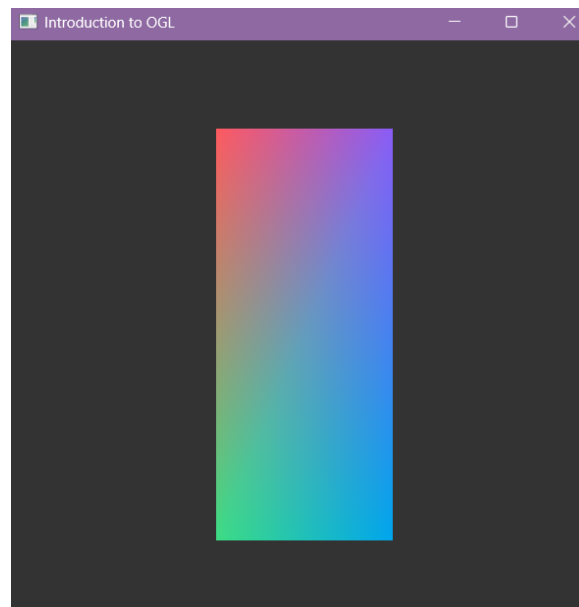


Figura 8: Ventana OpenGL con el rectángulo vertical.

Ahora se muestra el mismo rectángulo pero con el fondo de color azul:

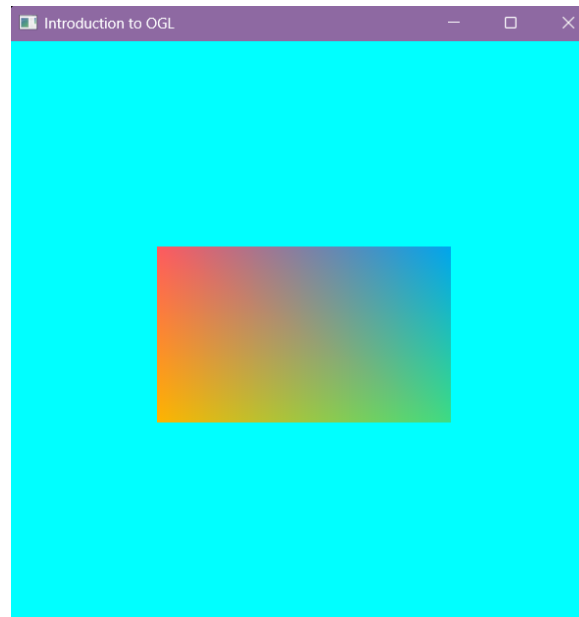


Figura 9: Ventana OpenGL con el rectángulo vertical y fondo azul.

4.4. Actividad 3: Medio círculo relleno

La tercera actividad consistió en desplegar en pantalla medio círculo relleno, empleando la clase `Circle` incluida en el proyecto base y sin recurrir a instrucciones de trazo directo. El reto principal era doble: por un lado, la clase genera la circunferencia completa, por lo que había que acotar la geometría a la mitad solicitada; por otro, los vértices resultantes no se conectan correctamente con la primitiva de dibujo que emplea el código base, lo que impedía obtener una superficie continua.

La solución se obtuvo mediante tres modificaciones al proyecto base. La primera se realizó en la función `Start()` del archivo principal, donde se comentó la construcción del triángulo y se habilitó en su lugar la instancia de la clase `Circle`, ajustando en consecuencia los argumentos con los que se construye el objeto `Mesh` para que recibiera los vértices e índices de la nueva geometría.

La segunda modificación se efectuó en el método `Draw()` de la clase `Mesh`, sustituyendo la primitiva `GL_TRIANGLES` por `GL_TRIANGLE_FAN`. El cambio responde a la manera en que la clase `Circle` genera su geometría: el primer vértice corresponde al centro de la figura y los restantes describen el contorno de forma consecutiva. Bajo ese orden, `GL_TRIANGLES` agrupa los vértices de tres en tres sin considerar el papel del centro, lo que producía una figura fragmentada; `GL_TRIANGLE_FAN`, en cambio, toma el primer vértice como pivote y forma un triángulo entre él y cada par de puntos contiguos del contorno, generando un abanico que rellena la superficie completa.

La tercera modificación se hizo en el archivo `Circle.h`, donde se redujo el límite del recorrido angular de 360° a 180° . Al conservar el incremento de 15° entre muestras, el trazo se detiene a la mitad de la circunferencia y el semicírculo queda conformado por doce triángulos a partir de trece vértices de contorno más el vértice central.

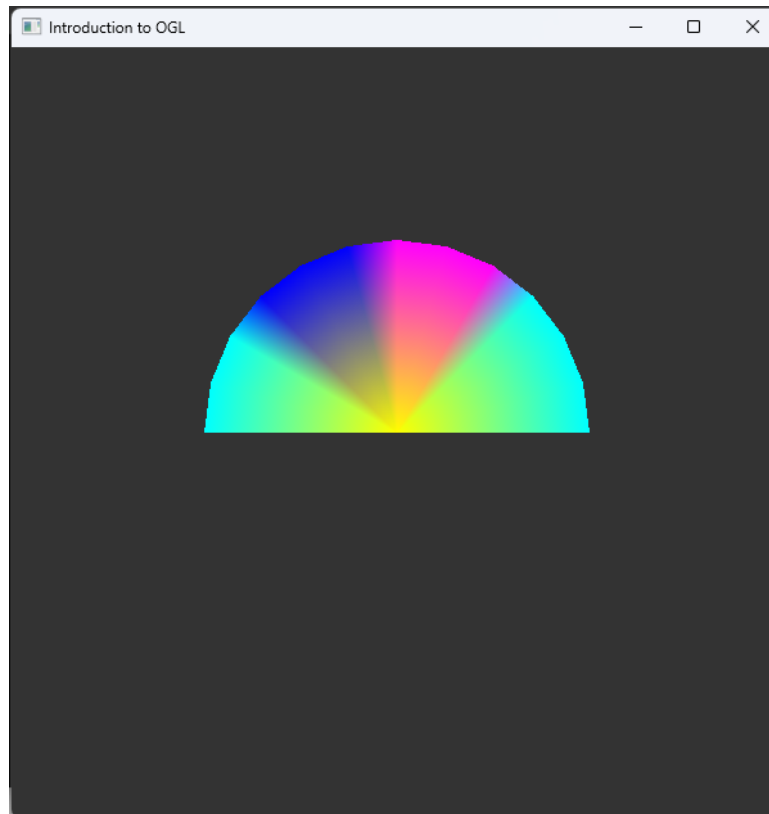


Figura 10: Medio círculo relleno generado con `GL_TRIANGLE_FAN`.

4.5. Actividad 4: Pentágono con colores distintos en cada vértice

Para esta actividad se creó la clase `Pentagon`, alojada en el archivo `include/Pentagon.h`, siguiendo la misma organización que emplean las geometrías del código base: un constructor que recibe el tamaño de la figura, los contenedores públicos `vertices` e `indices`, y los métodos privados `buildVertices` y `buildIndices`.

El primer paso fue convertir la paleta solicitada al formato normalizado que utiliza OpenGL. Cada canal del valor hexadecimal se interpreta como un entero de 8 bits, por lo que la conversión consiste en dividir dicho entero entre 255. La Tabla ?? concentra los cinco colores empleados.

Hexadecimal	R	G	B
#08F7FE	0.0314	0.9686	0.9961
#09FBD3	0.0353	0.9843	0.8275
#FE53BB	0.9961	0.3255	0.7333
#F5D300	0.9608	0.8275	0.0000
#7122FA	0.4431	0.1333	0.9804

Tabla 1: Conversión de la paleta de la actividad 4 al formato normalizado de OpenGL.

Los cinco vértices se calculan sobre una circunferencia de radio `size`. Al tratarse de un polígono regular de cinco lados, la separación angular entre vértices consecutivos es de $360^\circ/5 = 72^\circ$, y se tomó 90° como ángulo inicial para que el primer vértice quede en la parte superior de la ventana. La posición de cada vértice se obtiene entonces con $x = size \cdot \cos \theta$ y $y = size \cdot \sin \theta$.

Para obtener la figura rellena se generó el arreglo de índices mediante una triangulación en abanico a partir del primer vértice, es decir, los triángulos (0,1,2), (0,2,3) y (0,3,4). Esto permite cubrir el interior del pentágono reutilizando únicamente los cinco vértices de la frontera, de modo que cada uno conserva exactamente el color asignado y el interpolador del *fragment shader* se encarga del degradado. Como la clase entrega un vector de índices no vacío, la clase `Mesh` despliega la geometría con `glDrawElements` en modo `GL_TRIANGLES`.

```
1 int count = filled ? 5 : 6;
2
3 for (int i = 0; i < count; i++) {
4     Vertex v;
5     double rdeg = glm::radians(90.0 + 72.0 * (i % 5));
6     v.Position.x = size * (float)cos(rdeg);
7     v.Position.y = size * (float)sin(rdeg);
8     v.Position.z = 0.0f;
9     v.Color.r = palette[i % 5].r;
10    v.Color.g = palette[i % 5].g;
11    v.Color.b = palette[i % 5].b;
12    v.Color.a = 1.0f;
13    vertices.push_back(v);
14 }
```

Listing 6: Construcción de los vértices e índices del pentágono.

En la Figura ?? se muestra el pentágono relleno, donde se aprecia la interpolación de los cinco colores de la paleta a lo largo de la superficie.

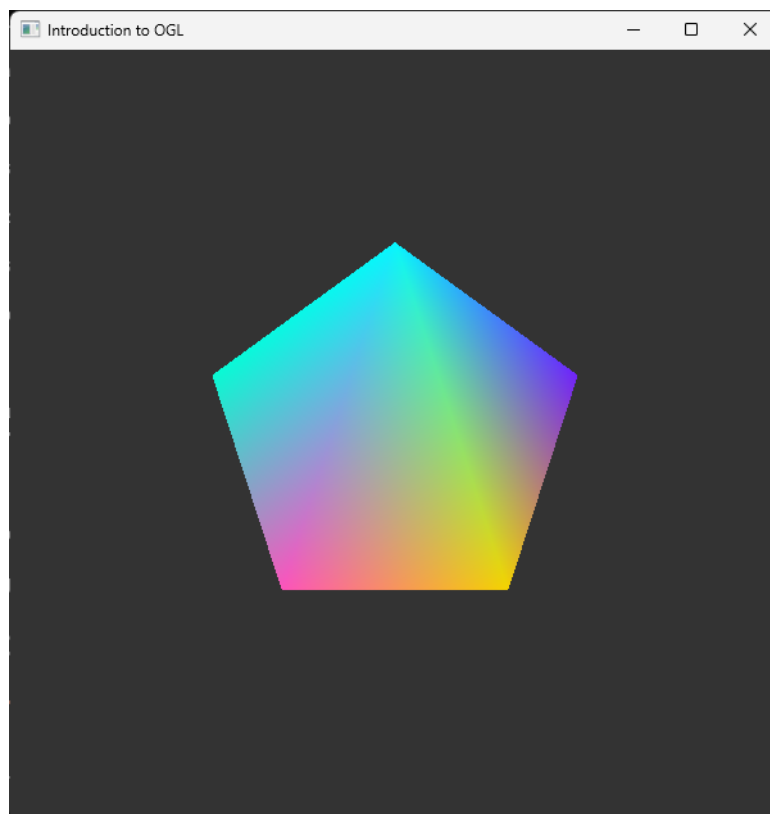


Figura 11: Pentágono regular relleno con un color distinto en cada vértice.

4.5.1. Variación de la actividad 4

Como variación se agregó al constructor el parámetro `filled`. Al construir la figura con `Pentagon pentagon(0.5f, false)` se omite la generación de índices y se repite el primer vértice al final del arreglo. Ante un vector de índices vacío, la clase `Mesh` recurre a `glDrawArrays` con el modo `GL_LINE_STRIP`, por lo que la misma geometría se despliega como contorno cerrado en lugar de una superficie rellena. El resultado se observa en la Figura ??.

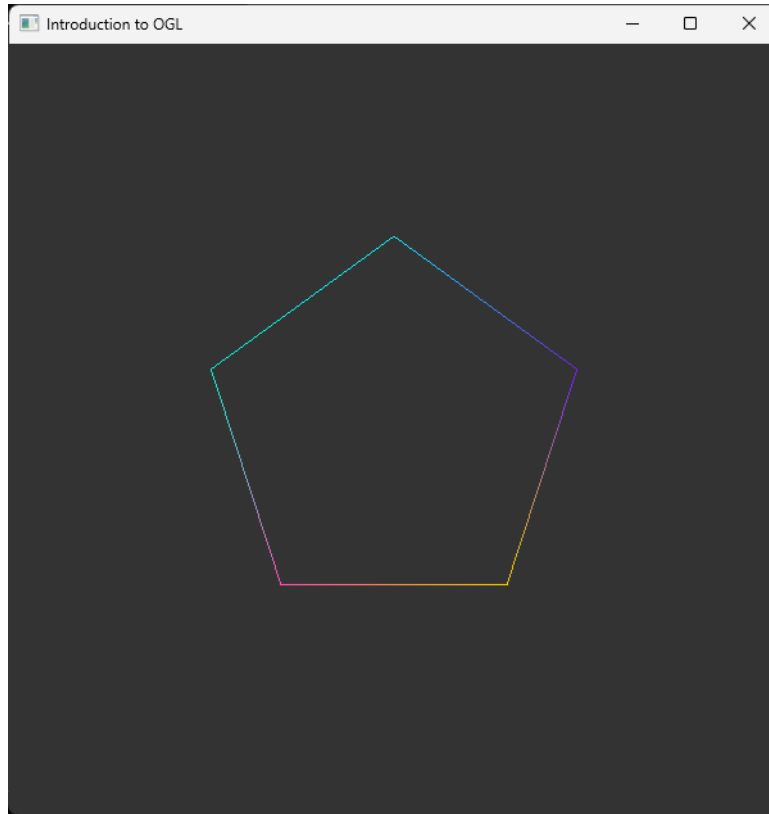


Figura 12: Variación de la actividad 4: el mismo pentágono desplegado como contorno.

4.6. Actividad 5: Función $\tan(2\theta)$

La última actividad consistió en graficar $\tan(2\theta)$ para $\theta = [-2\pi : 2\pi]$. El desarrollo se realizó sobre la clase `MathFunction`, que en el código base genera una función seno sobre un solo periodo, por lo que fue necesario replantear tanto el dominio como el tratamiento de la discontinuidad.

El primer aspecto por resolver fue el mapeo del dominio al espacio de coordenadas normalizadas de dispositivo, en el que la región visible está acotada al intervalo $[-1, 1]$ en ambos ejes. Dado que se requiere recorrer θ desde -2π hasta 2π , equivalente a $[-360^\circ, 360^\circ]$, la abscisa de cada vértice se obtiene como $x = \theta/2\pi$, lo que ocupa exactamente el ancho disponible de la ventana. La ordenada corresponde a la evaluación de la función escalada por el parámetro `size`.

El segundo aspecto es que la tangente no está definida cuando su argumento es un múltiplo impar de $\pi/2$. Para $\tan(2\theta)$ esto ocurre en $\theta = \pi/4 + k\pi/2$, de modo que dentro del dominio solicitado existen ocho asíntotas verticales que dividen la gráfica en nueve ramas continuas. Cerca de ellas la función crece sin cota y las muestras quedan fuera del área visible, por lo que se descartan aquellas cuya evaluación excede el intervalo $[-1, 1]$. Adicionalmente, se asignó un color por rama para distinguirlas con claridad, calculando su índice a partir del número de asíntotas que anteceden a cada muestra.


```

1 for (int i = -steps; i <= steps; i++) {
2
3     double deg = (double)i / samplesPerDegree;
4     float eval = size * (float)tan(2.0 * glm::radians(deg));
5
6     if (eval > 1.0f || eval < -1.0f)
7         continue;
8
9     Vertex v;
10    v.Position.x = (float)(deg / 360.0);
11    v.Position.y = eval;
12    v.Position.z = 0.0f;
13
14    vertices.push_back(v);
15 }

```

Listing 7: Evaluación y filtrado de las muestras de la función tangente.

El dominio se discretizó en cuartos de grado, lo que proporciona una resolución suficiente para que el trazo se perciba continuo pese a estar formado por segmentos de recta. Al no generarse índices, la clase `Mesh` despliega la geometría con `glDrawArrays` en modo `GL_LINE_STRIP`. Conviene señalar que este modo une todos los vértices en una sola polilínea, por lo que el último vértice de una rama queda enlazado con el primero de la siguiente; los segmentos casi verticales que se observan en la Figura ?? corresponden a esas uniones y coinciden en posición con las asíntotas de la función.

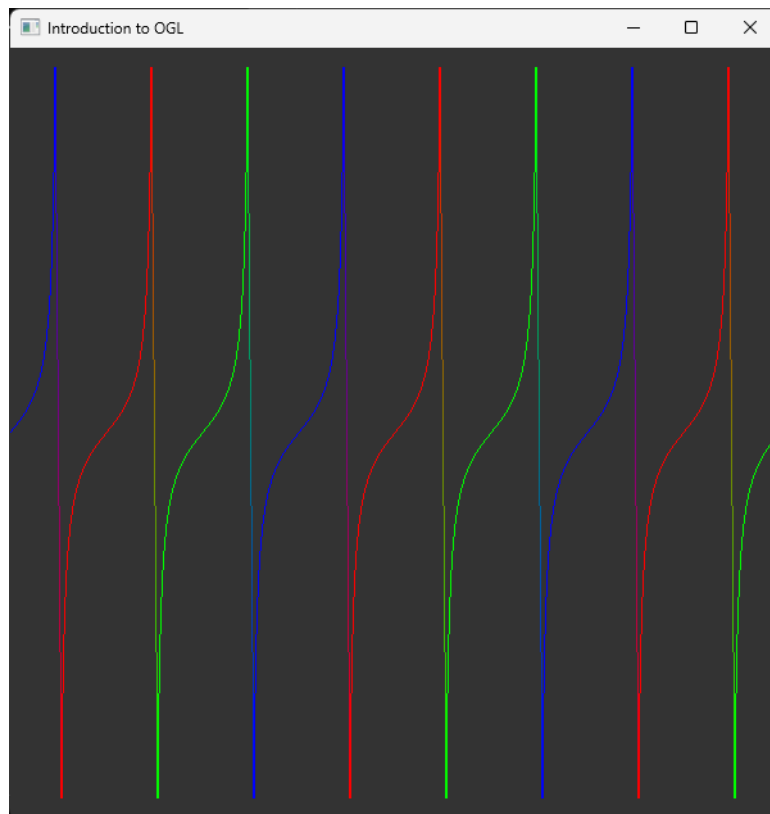


Figura 13: Función $\tan(2\theta)$ trazada para $\theta = [-2\pi : 2\pi]$.

4.6.1. Variación de la actividad 5

La variación consistió en modificar el factor de escala vertical con el que se construye la geometría, pasando de `MathFunction tanfunc(0.1f)` a `MathFunction tanfunc(0.5f)`. Al incrementar la

amplitud, la condición de recorte se satisface para valores de θ mucho más alejados de las asíntotas, de manera que la porción visible de cada rama se reduce a su tramo central y la curvatura característica de la tangente se pierde: las ramas aparecen como segmentos prácticamente rectos, tal como se muestra en la Figura ?? . El experimento evidencia que, en una función no acotada, el factor de escala determina qué parte de la curva alcanza a representarse dentro del área visible.

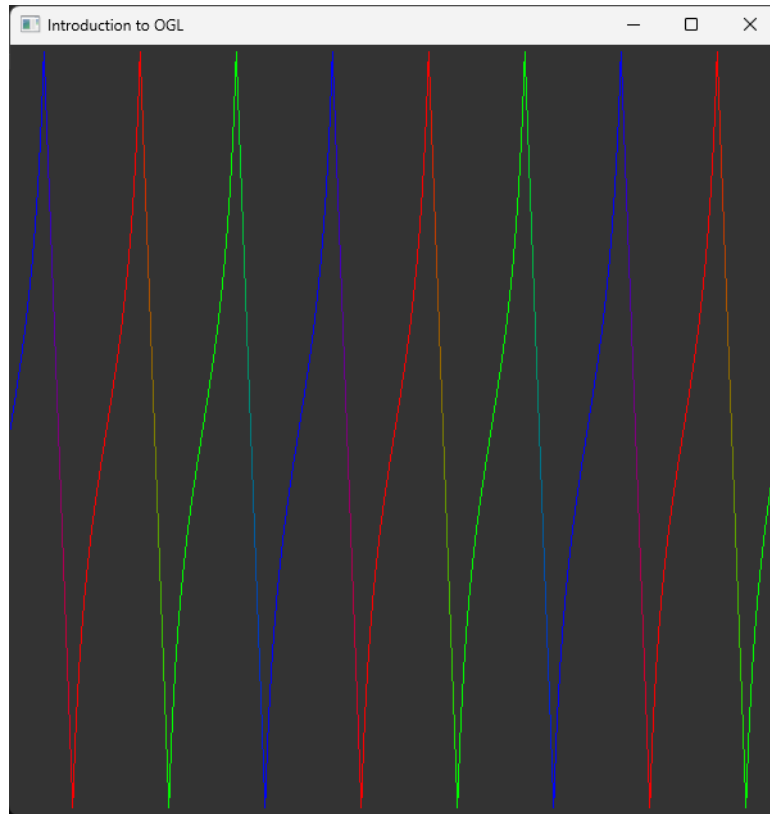


Figura 14: Variación de la actividad 5: efecto de aumentar el factor de escala vertical.

5. Resultados

5.1. Actividad 1

Tras la compilación y ejecución del programa, el resultado final fue una ventana gráfica capaz de alterar su color de fondo en tiempo real al presionar las teclas asignadas. Las pruebas en pantalla demostraron que la transición de colores ocurre de manera inmediata, logrando que el repintado del búfer coincida exactamente con los tonos de la paleta solicitada en las instrucciones de la práctica. Cabe destacar que las capturas de pantalla con los resultados finales en funcionamiento ya fueron presentadas y referenciadas en la sección de Experimentos.

Al evaluar este ejercicio, y cumpliendo con el análisis de la metodología requerida, se observaron los siguientes puntos sobre nuestro enfoque:

- **Ventajas del método:** La principal ventaja de realizar la conversión matemática a lápiz y alimentar a `glClearColor` directamente con valores flotantes normalizados es la optimización del rendimiento. El hardware de la tarjeta gráfica está diseñado para procesar números de punto flotante de manera nativa. Al entregarle los datos ya digeridos entre 0.0f y 1.0f, evitamos que el procesador desperdicie ciclos de reloj traduciendo cadenas de texto hexadecimales o enteros de 8 bits en cada iteración del bucle principal de renderizado.

- **Desventajas del método:** El punto débil de esta metodología es su rigidez a nivel de ingeniería de software. Al dejar los valores flotantes "quemados"(hardcoded) dentro de los condicionales de la función `keyboardCallback`, el código pierde escalabilidad. Si en el futuro se requiriera implementar una interfaz para que el usuario escoja un color personalizado, nuestro enfoque actual nos obligaría a recalcular a mano, alterar el código fuente y recompilar todo el proyecto. Una solución más elegante habría sido desarrollar una función auxiliar en C++ que parsee automáticamente un string hexadecimal (ej. "#00C2CB") y retorne un vector normalizado en tiempo de ejecución.

5.2. Actividad 2

Al compilar y ejecutar el programa, se logró observar en la pantalla un rectángulo horizontal que quedó centrado en el origen (0,0) sin deformarse. La figura se construyó uniendo dos triángulos mediante cuatro puntos para evitar repetir coordenadas. Cada esquina tomó de forma correcta el color asignado de la paleta al pasar los valores hexadecimales al formato decimal entre 0,0 y 1,0: coral (#FF5A5F) arriba a la izquierda, amarillo (#FFB400) abajo a la izquierda, verde (#3DDC84) abajo a la derecha y azul (#00A6ED) arriba a la derecha. En el centro de la figura no se presentaron cortes ni líneas, ya que la tarjeta gráfica se encargó de mezclar los tonos de forma suave.

Variación Actividad 2

Al modificar los valores de las posiciones en X y en Y , el rectángulo cambió a una orientación vertical manteniéndose centrado en la ventana. Asimismo, se integró el color morado (#8B5CF6) en la esquina superior derecha, combinándose de manera limpia con los demás tonos. Finalmente, al cambiar el orden de conexión de los índices para trazar una diagonal diferente entre los dos triángulos, el degradado interior se adaptó a la nueva división, comprobando de forma práctica cómo influye la unión de los puntos en la distribución final del color.

Observaciones: Ventajas y Desventajas

Trabajar de esta forma con OpenGL tiene varios puntos a favor y en contra que pude notar al hacer la actividad:

- **Ventajas:**
 - **Ahorro de memoria y datos:** Usar una lista de índices nos evita tener que escribir y guardar seis vértices con sus colores repetidos; con solo definir cuatro puntos se arman los dos triángulos sin problema.
 - **Interpolación automática:** No hace falta calcular a mano cómo se mezclan los colores punto por punto, ya que la tarjeta gráfica (GPU) se encarga de hacer el degradado suave y en tiempo real.
 - **Fácil modificación:** Cambiar el tamaño, la posición o los colores de la figura es muy directo porque solo se modifican los valores de los vértices sin tener que alterar la lógica del programa.
- **Desventajas:**
 - **Dependencia del orden de los índices:** El degradado interior no siempre queda simétrico, ya que depende totalmente de por dónde pase la diagonal que divide a los triángulos.
 - **Complejidad inicial:** Para dibujar una figura básica como un rectángulo se requiere configurar bastantes cosas previas (coordenadas normalizadas, arreglos de memoria, shaders y conversiones de color), lo cual puede ser confuso al inicio.

5.3. Actividad 3

Se obtuvo un semicírculo relleno centrado en el origen, cuyo resultado en pantalla se muestra en la Figura ?? de la sección de Experimentos. El trazo aparece cerrado sobre el diámetro y sin discontinuidades en el arco, lo que confirma que el recorrido angular limitado a 180° genera la mitad exacta de la circunferencia.

El resultado más relevante de esta actividad fue comprobar que una misma lista de vértices produce figuras distintas según la primitiva con que se interprete. Con `GL_TRIANGLES` la geometría se desplegaba fragmentada, ya que los vértices se agrupan de tres en tres sin considerar el vértice central; al emplear `GL_TRIANGLE_FAN` ese primer vértice pasa a funcionar como pivote común y la superficie se rellena por completo. La corrección no requirió modificar una sola coordenada, únicamente el criterio de conexión entre los vértices ya existentes.

Cabe señalar que la resolución de la figura queda determinada por el incremento angular. Al conservarse el paso de 15° , el contorno se aproxima mediante doce segmentos rectos, lo cual resulta perceptible en el borde curvo; reducir dicho incremento suavizaría el arco a costa de generar un mayor número de vértices.

5.4. Actividad 4

Se obtuvo un pentágono regular relleno, centrado en el origen y con los cinco colores de la paleta solicitada asignados uno por vértice. La conversión de los valores hexadecimales al formato normalizado se verificó de manera visual comparando el tono de cada vértice contra la paleta del manual. El degradado observado en el interior de la figura confirma que la interpolación de los atributos de color ocurre en la etapa de rasterización, entre el *vertex shader* y el *fragment shader*, sin necesidad de declarar color alguno por fragmento.

La variación mostró que la representación de una misma geometría depende del arreglo de índices: al suprimirlo, los cinco vértices se despliegan como una polilínea cerrada en lugar de tres triángulos, sin modificar una sola coordenada.

5.5. Actividad 5

La gráfica de $\tan(2\theta)$ se desplegó completa sobre el dominio $[-2\pi : 2\pi]$, con sus nueve ramas continuas separadas por las ocho asíntotas verticales previstas analíticamente en $\theta = \pi/4 + k\pi/2$. El coloreado por rama permitió comprobar que la segmentación obtenida coincide con la esperada.

El recorte de las muestras que exceden el intervalo visible resultó indispensable, ya que sin él los vértices cercanos a las asíntotas se proyectan a valores arbitrariamente grandes. La variación del factor de escala evidenció que dicho parámetro no altera la función, sino la porción de cada rama que alcanza a mostrarse antes de salir del área de dibujo.

6. Conclusiones

6.1. Basilio Illescas Andrés

El desarrollo de las actividades asignadas dejó ver que la construcción de una geometría en OpenGL se apoya en dos decisiones independientes entre sí: cuáles son los vértices que la definen y de qué manera se conectan. En el caso del pentágono, los cinco vértices calculados sobre una circunferencia bastaron

para obtener tanto la figura rellena como su contorno; lo único que cambió entre un resultado y otro fue la presencia del arreglo de índices, que determina si la malla se despliega con `glDrawElements` en modo `GL_TRIANGLES` o con `glDrawArrays` en modo `GL_LINE_STRIP`. Igualmente resultó ilustrativo que el degradado del interior de la figura no requiera código adicional: al declarar el color como un atributo por vértice, la interpolación queda a cargo de la etapa de rasterización.

La actividad de la función tangente mostró que trasladar una expresión matemática a la pantalla obliga a resolver dos problemas ajenos a la expresión misma. El primero es el mapeo del dominio y del codominio al espacio de coordenadas normalizadas de dispositivo, que limita la región visible al intervalo $[-1, 1]$. El segundo es el tratamiento de los valores que no pueden representarse: el pipeline gráfico no tiene noción de discontinuidad y une en una sola polilínea todos los vértices que recibe, de modo que las asíntotas de $\tan(2\theta)$ tuvieron que atenderse de forma explícita al construir la geometría, filtrando las muestras que quedan fuera del área de dibujo.

Finalmente, la conversión de los colores hexadecimales al formato normalizado reforzó la idea de que OpenGL opera sobre valores flotantes acotados y no sobre las representaciones enteras habituales en otros contextos, criterio que se repite tanto en la definición del color de los vértices como en la del color de fondo de la ventana.

En lo que corresponde a las actividades desarrolladas, el objetivo de generar un programa gráfico capaz de desplegar figuras en dos dimensiones se cumplió: ambas geometrías se construyeron a partir de las instrucciones básicas de la API, especificando su información en arreglos de vértices e índices que se cargan en los *buffers* correspondientes y delegando el despliegue al par de *shaders* del proyecto base. Conviene precisar que estas dos actividades no requirieron atender eventos de entrada, por lo que el objetivo relativo al manejo de *callbacks* se abordó en las actividades restantes de la práctica.

6.2. Cruz Macedo Samuel Santiago

Esta primera práctica sirvió para dimensionar la cantidad de elementos que deben coordinarse antes de que aparezca la figura más sencilla en pantalla. Antes de dibujar un solo vértice fue necesario inicializar la biblioteca de ventanas, solicitar un contexto de OpenGL de la versión adecuada, cargar los apuntadores a las funciones de la API y compilar el par de programas que se ejecutan en la tarjeta gráfica. Comprender que ese andamiaje es un requisito y no un accesorio ayudó a interpretar la estructura del proyecto base, en la que las funciones `Start` y `Update` separan con claridad lo que se configura una sola vez de lo que se repite en cada cuadro.

El punto que resultó más provechoso fue entender el papel de los *Vertex Array Objects* y los *Vertex Buffer Objects* como el mecanismo mediante el cual la información de una geometría viaja de la memoria del programa a la de la tarjeta gráfica, y cómo la descripción de los atributos determina la forma en que el *vertex shader* interpreta esos datos. A partir de ello se vuelve evidente que las figuras desplegadas durante la práctica no se dibujan con instrucciones de trazo, sino declarando la información de sus vértices y el modo en que deben conectarse.

En conjunto, se considera alcanzado el objetivo de la práctica, ya que se logró generar un programa gráfico funcional que despliega figuras en dos dimensiones empleando las instrucciones básicas de la API, y quedaron identificados los elementos que intervienen en el proceso.

6.3. Medel Piña Ángel de Jesús

Trabajar en la construcción del semicírculo me hizo notar que en OpenGL no basta con tener los vértices correctos: hace falta indicarle a la API bajo qué criterio debe unirlos. La primera ejecución del programa devolvió una figura fragmentada aun cuando las coordenadas ya estaban bien calculadas, y el problema se resolvió únicamente cambiando la primitiva de dibujo de `GL_TRIANGLES` a `GL_TRIANGLE_FAN`. Ese detalle me pareció el aprendizaje más claro de mi actividad, porque muestra que la geometría y la forma de recorrerla son dos cosas distintas, y que un error en la segunda puede aparentar ser un error en la primera.

También resultó útil entender por qué esa primitiva era la adecuada. Como la clase `Circle` coloca el centro como primer vértice y después genera el contorno de manera consecutiva, `GL_TRIANGLE_FAN` aprovecha exactamente ese orden: toma el primer vértice como pivote y arma un abanico de triángulos hacia cada par de puntos contiguos. Reconocer la relación entre cómo se generan los vértices y qué modo de dibujo los interpreta correctamente me pareció más valioso que el resultado visual en sí.

Por último, limitar el recorrido angular de 360° a 180° evidenció que el nivel de detalle de una figura curva depende por completo de su discretización. El arco no es realmente una curva, sino una sucesión de segmentos rectos determinada por el incremento del ángulo, de modo que existe un compromiso directo entre la suavidad del contorno y la cantidad de vértices que se envían a la tarjeta gráfica.

6.4. Perez Olvera Alexis Abraham

Esta primera inmersión práctica en OpenGL representó un puente crucial entre la lógica de programación pura y la interacción directa con el hardware gráfico. En su mayoría nos solemos enfocarnos en el procesamiento en la CPU, pero configurar manualmente la memoria a través de los VAO (Vertex Array Objects) y VBO (Vertex Buffer Objects) me permitió entender cómo se estructuran y envían los datos a la tarjeta de video.

En particular, el desarrollo del primer ejercicio me ayudó a visualizar la importancia del ciclo continuo de renderizado. Procesar las interrupciones físicas del teclado mediante callbacks y transformar esos eventos en operaciones matemáticas —como la normalización de colores hexadecimales a valores flotantes para la función `glClearColor`— demostró cómo convergen el software y los periféricos de entrada/salida en tiempo real. En definitiva, la práctica sentó bases muy sólidas para dejar de ver los gráficos como simples trazos en pantalla y empezar a comprenderlos como vértices, búferes y matrices operando a bajo nivel.

6.5. Segura Cedeño Luisa María

Esta práctica me ayudó a entender cómo funciona OpenGL desde cero y cómo se crean figuras básicas. Al principio la configuración de todo el entorno y las herramientas fue bastante pesada y complicada, sobre todo porque mi computadora se puso muy lenta durante las instalaciones y me tomó mucho tiempo dejar todo listo para poder empezar.

Una vez que quedó configurado, me enfoqué en trabajar con la **Actividad 2**, donde logré dibujar el rectángulo centrado usando dos triángulos unidos. Lo más interesante fue ver cómo al asignarle a cada esquina un color diferente (pasando los valores hexadecimales a números entre 0 y 1), la propia tarjeta gráfica se encargó de mezclar los tonos en el centro para que se viera un degradado continuo.

También al hacer la variación y cambiar el orden de las conexiones y el tamaño, pude comprobar cómo cualquier cambio en los puntos afecta directamente la forma y los colores finales en la ventana. En general, aunque la instalación costó trabajo por el rendimiento del equipo, la práctica me sirvió bastante para entender cómo se manejan las figuras y los colores en pantalla.

Referencias

- [1] Microsoft Learn. (s.f.). *Control gráfico OpenGL - Win32 apps*. Recuperado de <https://learn.microsoft.com/es-es/windows/win32/opengl/introduction-to-opengl>
- [2] Convert Color Code. (s.f.). *Convertidor de códigos de color HEXA a RGBA*. Recuperado de <https://convertcolorcode.com/es/convertir/hexa/rgba>
- [3] Microsoft Learn. (s.f.). *Modo RGBA y administración de paletas de Windows - Win32 apps*. Recuperado de <https://learn.microsoft.com/es-es/windows/win32/opengl/rgba-mode-and-windows-palette-management>
- [4] Universitat de València. (s.f.). *Texturas en OpenGL*. Recuperado de <https://informatica.uv.es/iigua/AIG/docs/texturas.htm>